

```

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import ResNet50, VGG16
from tensorflow.keras.preprocessing import image_dataset_from_directory
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import numpy as np
import os
import cv2

```

```

from google.colab import files
uploaded = files.upload()

```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving spiral zin to spiral zin

```

from tensorflow.keras.applications import InceptionV3
from tensorflow.keras import layers, models, optimizers

def build_and_train_inceptionv3(train_ds, test_ds, epochs=10):
    """Builds, compiles, trains, and evaluates an InceptionV3 model."""
    base_model = InceptionV3(weights='imagenet', include_top=False, input_shape=
    base_model.trainable = False

    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(3, activation='softmax')
    ])

    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metr

    print("\nTraining InceptionV3 model...")
    model.fit(train_ds, epochs=epochs, verbose=0)

    print("Evaluating InceptionV3 model...")
    loss, acc = model.evaluate(test_ds, verbose=0)
    return acc

acc_inceptionv3 = build_and_train_inceptionv3(train_ds, test_ds)
print(f'InceptionV3 Test Accuracy: {acc_inceptionv3:.4f}')

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/87910968/87910968>  5s 0us/step

```

Training InceptionV3 model...
Evaluating InceptionV3 model...
InceptionV3 Test Accuracy: 0.7667

```

```

from tensorflow.keras.applications import DenseNet121
from tensorflow.keras import layers, models, optimizers

def build_and_train_densenet121(train_ds, test_ds, epochs=20):
    """Builds, compiles, trains, and evaluates a DenseNet121 model."""
    base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=
    base_model.trainable = False

    model = models.Sequential([
        base_model,
        layers.GlobalAveragePooling2D(),
        layers.Dense(2, activation='softmax')
    ])

    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metr

    print("\nTraining DenseNet121 model...")
    model.fit(train_ds, epochs=epochs, verbose=0)

    print("Evaluating DenseNet121 model...")
    loss, acc = model.evaluate(test_ds, verbose=0)
    return acc

acc_densenet121 = build_and_train_densenet121(train_ds, test_ds)
print(f'DenseNet121 Test Accuracy: {acc_densenet121:.4f}')

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet121/densenet121_weights_tf_dim_ordering_tf_data_format.h5 29084464/29084464 ————— 2s 0us/step

Training DenseNet121 model...
 Evaluating DenseNet121 model...
 DenseNet121 Test Accuracy: 0.8333

```

import zipfile, os

with zipfile.ZipFile("spiral.zip", 'r') as zip_ref:
    zip_ref.extractall("spiral")

os.listdir("spiral")

['spiral']

```

```

import os
os.listdir('/content')

['.config', 'spiral', 'spiral.zip', 'sample_data']

```

```

import os
os.listdir('/content/spiral/spiral')

```

```
['testing', 'training']
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing import image_dataset_from_directory
import numpy as np
import os, cv2, zipfile

if os.path.exists("/content/spiral.zip"):
    with zipfile.ZipFile("/content/spiral.zip", 'r') as zip_ref:
        zip_ref.extractall("/content/spiral")
    print(" Extracted spiral.zip successfully!")

print("Root directory:", os.listdir('/content/spiral'))

if 'spiral' in os.listdir('/content/spiral'):
    dataset_path = '/content/spiral/spiral'
else:
    dataset_path = '/content/spiral'

print("Using dataset path:", dataset_path)
print("Subfolders:", os.listdir(dataset_path))

train_ds = image_dataset_from_directory(
    os.path.join(dataset_path, 'training'),
    image_size=(224, 224),
    batch_size=32
)
test_ds = image_dataset_from_directory(
    os.path.join(dataset_path, 'testing'),
    image_size=(224, 224),
    batch_size=32
)

print("Classes in DL loader:", train_ds.class_names)

train_ds = train_ds.map(lambda x, y: (x / 255.0, y))
test_ds = test_ds.map(lambda x, y: (x / 255.0, y))

def load_images(folder):
    images = []
    labels = []
    for label, subfolder in enumerate(['healthy', 'parkinson']):
        path = os.path.join(folder, subfolder)
        for file in os.listdir(path):
            img_path = os.path.join(path, file)
            if file.lower().endswith(('.png', '.jpg', '.jpeg')):
                img = cv2.imread(img_path)
                img = cv2.resize(img, (224, 224))
                img = img / 255.0 # Normalize
                images.append(img.flatten())
                labels.append(label)
    return np.array(images), np.array(labels)
```

```
X_train_ml, y_train_ml = load_images(os.path.join(dataset_path, 'training'))
X_test_ml, y_test_ml = load_images(os.path.join(dataset_path, 'testing'))

print(" Data Loaded Successfully")
print("Train shape:", X_train_ml.shape, "Labels:", y_train_ml.shape)
print("Test shape:", X_test_ml.shape, "Labels:", y_test_ml.shape)
```

```
Extracted spiral.zip successfully!
Root directory: ['spiral']
Using dataset path: /content/spiral/spiral
Subfolders: ['testing', 'training']
Found 72 files belonging to 2 classes.
Found 30 files belonging to 2 classes.
Classes in DL loader: ['healthy', 'parkinson']
Data Loaded Successfully
Train shape: (72, 150528) Labels: (72,)
Test shape: (30, 150528) Labels: (30,)
```

```
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(2, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
model.fit(train_ds, epochs=10)
loss, acc = model.evaluate(test_ds)
print(f'Custom CNN Accuracy: {acc:.4f}')
```

```
Epoch 1/10
3/3 ————— 7s 830ms/step - accuracy: 0.4193 - loss: 9.1289
Epoch 2/10
3/3 ————— 0s 42ms/step - accuracy: 0.5256 - loss: 0.7712
Epoch 3/10
3/3 ————— 0s 44ms/step - accuracy: 0.5234 - loss: 0.7116
Epoch 4/10
3/3 ————— 0s 42ms/step - accuracy: 0.4666 - loss: 0.7033
Epoch 5/10
3/3 ————— 0s 37ms/step - accuracy: 0.4766 - loss: 0.6922
Epoch 6/10
3/3 ————— 0s 39ms/step - accuracy: 0.4688 - loss: 0.6934
Epoch 7/10
3/3 ————— 0s 36ms/step - accuracy: 0.4648 - loss: 0.6952
Epoch 8/10
3/3 ————— 0s 39ms/step - accuracy: 0.5039 - loss: 0.6912
Epoch 9/10
3/3 ————— 0s 40ms/step - accuracy: 0.4883 - loss: 0.6896
Epoch 10/10
3/3 ————— 0s 45ms/step - accuracy: 0.5434 - loss: 0.6895
1/1 ————— 1s 529ms/step - accuracy: 0.5000 - loss: 0.6919
Custom CNN Accuracy: 0.5000
```

```

base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(2, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(train_ds, epochs=15)
loss, acc = model.evaluate(test_ds)
print(f'ResNet50 Accuracy: {acc:.4f}')

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5 94765736/94765736 ————— 5s 0us/step

```

Epoch 1/15
3/3 ————— 16s 2s/step - accuracy: 0.4844 - loss: 0.9997
Epoch 2/15
3/3 ————— 0s 60ms/step - accuracy: 0.4627 - loss: 0.7368
Epoch 3/15
3/3 ————— 0s 62ms/step - accuracy: 0.5273 - loss: 0.7021
Epoch 4/15
3/3 ————— 0s 60ms/step - accuracy: 0.5234 - loss: 0.7087
Epoch 5/15
3/3 ————— 0s 61ms/step - accuracy: 0.5078 - loss: 0.6940
Epoch 6/15
3/3 ————— 0s 59ms/step - accuracy: 0.6545 - loss: 0.6824
Epoch 7/15
3/3 ————— 0s 64ms/step - accuracy: 0.5273 - loss: 0.6932
Epoch 8/15
3/3 ————— 0s 60ms/step - accuracy: 0.4844 - loss: 0.7068
Epoch 9/15
3/3 ————— 0s 60ms/step - accuracy: 0.5100 - loss: 0.6891
Epoch 10/15
3/3 ————— 0s 61ms/step - accuracy: 0.6506 - loss: 0.6827
Epoch 11/15
3/3 ————— 0s 64ms/step - accuracy: 0.4844 - loss: 0.6813
Epoch 12/15
3/3 ————— 0s 61ms/step - accuracy: 0.5334 - loss: 0.6797
Epoch 13/15
3/3 ————— 0s 61ms/step - accuracy: 0.5985 - loss: 0.6754
Epoch 14/15
3/3 ————— 0s 62ms/step - accuracy: 0.6445 - loss: 0.6773
Epoch 15/15
3/3 ————— 0s 62ms/step - accuracy: 0.5347 - loss: 0.6839
1/1 ————— 7s 7s/step - accuracy: 0.5667 - loss: 0.6768
ResNet50 Accuracy: 0.5667

```

```

base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(3, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(train_ds, epochs=10)

```

```
loss, acc = model.evaluate(test_ds)
print(f'VGG16 Accuracy: {acc:.4f}')
```

```
Epoch 1/10
3/3 ————— 4s 496ms/step - accuracy: 0.0247 - loss: 1.2536
Epoch 2/10
3/3 ————— 0s 107ms/step - accuracy: 0.4774 - loss: 1.0793
Epoch 3/10
3/3 ————— 0s 111ms/step - accuracy: 0.5256 - loss: 0.9519
Epoch 4/10
3/3 ————— 0s 112ms/step - accuracy: 0.6072 - loss: 0.8594
Epoch 5/10
3/3 ————— 0s 112ms/step - accuracy: 0.6849 - loss: 0.7920
Epoch 6/10
3/3 ————— 0s 113ms/step - accuracy: 0.7865 - loss: 0.7454
Epoch 7/10
3/3 ————— 0s 110ms/step - accuracy: 0.7904 - loss: 0.7159
Epoch 8/10
3/3 ————— 0s 106ms/step - accuracy: 0.7943 - loss: 0.6820
Epoch 9/10
3/3 ————— 0s 109ms/step - accuracy: 0.8021 - loss: 0.6558
Epoch 10/10
3/3 ————— 0s 109ms/step - accuracy: 0.7982 - loss: 0.6367
1/1 ————— 1s 978ms/step - accuracy: 0.7667 - loss: 0.6217
VGG16 Accuracy: 0.7667
```

```
svm = SVC()
svm.fit(X_train_ml, y_train_ml)
y_pred = svm.predict(X_test_ml)
acc = accuracy_score(y_test_ml, y_pred)
print(f'SVM Accuracy: {acc:.4f}')
```

SVM Accuracy: 0.8000

```
knn = KNeighborsClassifier()
knn.fit(X_train_ml, y_train_ml)
y_pred = knn.predict(X_test_ml)
acc = accuracy_score(y_test_ml, y_pred)
print(f'KNN Accuracy: {acc:.4f}')
```

KNN Accuracy: 0.4667

```
dt = DecisionTreeClassifier()
dt.fit(X_train_ml, y_train_ml)
y_pred = dt.predict(X_test_ml)
acc = accuracy_score(y_test_ml, y_pred)
print(f'Decision Tree Accuracy: {acc:.4f}')
```

Decision Tree Accuracy: 0.6000

```
from tensorflow.keras.applications import DenseNet121, InceptionV3
from tensorflow.keras import optimizers
```

```
def build_and_train_gwo_model(base_model_architecture, hyperparameters, train_ds
```

```

"""Builds, compiles, and trains a GWO-based model."""

if base_model_architecture == 'VGG16':
    base_model = VGG16(weights='imagenet', include_top=False, input_shape=(2
elif base_model_architecture == 'DenseNet':
    base_model = DenseNet121(weights='imagenet', include_top=False, input_sh
elif base_model_architecture == 'InceptionV3':
    base_model = InceptionV3(weights='imagenet', include_top=False, input_sh
else:
    raise ValueError("Unsupported base model architecture")

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(hyperparameters['dense_units'], activation='relu'),
    layers.Dropout(hyperparameters['dropout_rate']),
    layers.Dense(2, activation='softmax')
])

optimizer = optimizers.Adam(learning_rate=hyperparameters['learning_rate'])

model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy', m

history = model.fit(train_ds, epochs=hyperparameters['epochs'], validation_d

loss, acc = model.evaluate(test_ds, verbose=0)
return acc, history.history['val_accuracy'][-1]

```

```

import random

def grey_wolf_optimizer(model_architecture, train_ds, test_ds, num_wolves=5, num
    """Implements the Grey Wolf Optimizer to find optimal hyperparameters."""

    wolves = []
    for _ in range(num_wolves):
        wolf_hyperparameters = {
            'learning_rate': random.uniform(hyperparameter_bounds['learning_rate
            'dense_units': random.randint(hyperparameter_bounds['dense_units']
            'dropout_rate': random.uniform(hyperparameter_bounds['dropout_rate']
            'epochs': random.randint(hyperparameter_bounds['epochs'][0], hyperpa
        }
        wolves.append({'hyperparameters': wolf_hyperparameters, 'fitness': -floa

    alpha_wolf = {'fitness': -float('inf')}
    beta_wolf = {'fitness': -float('inf')}
    delta_wolf = {'fitness': -float('inf')}

    for iteration in range(num_iterations):
        print(f"GWO Iteration {iteration + 1}/{num_iterations}")
        for wolf in wolves:
            try:

```

```

_, val_acc = build_and_train_gwo_model(model_architecture, wolf[
wolf['fitness'] = val_acc
except Exception as e:
    print(f"Error training model with hyperparameters {wolf['hyperpa
wolf['fitness'] = -float('inf')
if wolf['fitness'] > alpha_wolf['fitness']:
    alpha_wolf = wolf.copy()
elif wolf['fitness'] > beta_wolf['fitness']:
    beta_wolf = wolf.copy()
elif wolf['fitness'] > delta_wolf['fitness']:
    delta_wolf = wolf.copy()

a = 2 - iteration * (2 / num_iterations)
for wolf in wolves:

    for param in wolf['hyperparameters']:
        r1 = random.random()
        r2 = random.random()
        A1 = 2 * a * r1 - a
        C1 = 2 * r2
        D_alpha = abs(C1 * alpha_wolf['hyperparameters'][param] - wolf['f
        X1 = alpha_wolf['hyperparameters'][param] - A1 * D_alpha

        r1 = random.random()
        r2 = random.random()
        A2 = 2 * a * r1 - a
        C2 = 2 * r2
        D_beta = abs(C2 * beta_wolf['hyperparameters'][param] - wolf['hy
        X2 = beta_wolf['hyperparameters'][param] - A2 * D_beta

        r1 = random.random()
        r2 = random.random()
        A3 = 2 * a * r1 - a
        C3 = 2 * r2
        D_delta = abs(C3 * delta_wolf['hyperparameters'][param] - wolf['f
        X3 = delta_wolf['hyperparameters'][param] - A3 * D_delta

    wolf['hyperparameters'][param] = (X1 + X2 + X3) / 3
    if param in hyperparameter_bounds:
        wolf['hyperparameters'][param] = max(hyperparameter_bounds[p
    if isinstance(hyperparameter_bounds[param][0], int):
        wolf['hyperparameters'][param] = int(round(wolf['hyperparam

return alpha_wolf['hyperparameters']

```

```

from tensorflow.keras.applications import VGG19, ResNet50
from tensorflow.keras import layers, models, optimizers

def build_and_train_model(base_model_architecture, train_ds, test_ds, epochs=10)
    """Builds, compiles, trains, and evaluates a given model."""

    if base_model_architecture == 'VGG19':

```



```

base_model = VGG19(weights='imagenet', include_top=False, input_shape=(2
elif base_model_architecture in ['ResNet18', 'ResNet101']:

    print(f"Using ResNet50 as a base for {base_model_architecture} (Note: Th
    base_model = ResNet50(weights='imagenet', include_top=False, input_shape
else:
    raise ValueError("Unsupported base model architecture")

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(2, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metr

print(f"\nTraining {base_model_architecture} model...")
model.fit(train_ds, epochs=epochs, verbose=0)

print(f"Evaluating {base_model_architecture} model...")
loss, acc = model.evaluate(test_ds, verbose=0)
return acc

acc_vgg19 = build_and_train_model('VGG19', train_ds, test_ds)
print(f'VGG19 Test Accuracy: {acc_vgg19:.4f}')

acc_resnet18 = build_and_train_model('ResNet18', train_ds, test_ds)
print(f'ResNet18 (using ResNet50 base) Test Accuracy: {acc_resnet18:.4f}')

acc_resnet101 = build_and_train_model('ResNet101', train_ds, test_ds)
print(f'ResNet101 (using ResNet50 base) Test Accuracy: {acc_resnet101:.4f}')

```

Downloading data from [https://storage.googleapis.com/tensorflow/keras-applications:80134624/80134624](https://storage.googleapis.com/tensorflow/keras-applications/80134624/80134624)  5s 0us/step

Training VGG19 model...
 Evaluating VGG19 model...
 VGG19 Test Accuracy: 0.5333
 Using ResNet50 as a base for ResNet18 (Note: This is a simplification).

Training ResNet18 model...
 Evaluating ResNet18 model...
 ResNet18 (using ResNet50 base) Test Accuracy: 0.5333
 Using ResNet50 as a base for ResNet101 (Note: This is a simplification).

Training ResNet101 model...
 Evaluating ResNet101 model...
 ResNet101 (using ResNet50 base) Test Accuracy: 0.6000

```

from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.layers import LSTM, TimeDistributed, Flatten
from tensorflow.keras import models, optimizers

```

```

base_model = DenseNet121(weights='imagenet', include_top=False, input_shape=(224
base_model.trainable = False

model = models.Sequential([
    base_model,
    TimeDistributed(Flatten()),
    LSTM(128),
    layers.Dense(2, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=

print("Training DenseNet+LSTM model...")
model.fit(train_ds, epochs=20)

print("Evaluating DenseNet+LSTM model...")
loss, acc = model.evaluate(test_ds)
print(f'DenseNet+LSTM Test Accuracy: {acc:.4f}')

```

Training DenseNet+LSTM model...

Epoch 1/20

3/3  15s 102ms/step - accuracy: 0.4744 - loss: 1.5687

Epoch 2/20

3/3  0s 98ms/step - accuracy: 0.4905 - loss: 0.9189

Epoch 3/20

3/3  0s 97ms/step - accuracy: 0.5286 - loss: 0.6298

Epoch 4/20

3/3  0s 98ms/step - accuracy: 0.6155 - loss: 0.6012

Epoch 5/20

3/3  0s 98ms/step - accuracy: 0.7856 - loss: 0.5018

Epoch 6/20

3/3  0s 97ms/step - accuracy: 0.7943 - loss: 0.4618

Epoch 7/20

3/3  0s 99ms/step - accuracy: 0.8247 - loss: 0.4527

Epoch 8/20

3/3  0s 98ms/step - accuracy: 0.8207 - loss: 0.4130

Epoch 9/20

3/3  0s 98ms/step - accuracy: 0.8260 - loss: 0.4045

Epoch 10/20

3/3  0s 94ms/step - accuracy: 0.8672 - loss: 0.3728

Epoch 11/20

3/3  0s 94ms/step - accuracy: 0.8859 - loss: 0.3526

Epoch 12/20

3/3  0s 100ms/step - accuracy: 0.9154 - loss: 0.3240

Epoch 13/20

3/3  0s 97ms/step - accuracy: 0.9301 - loss: 0.2851

Epoch 14/20

3/3  0s 111ms/step - accuracy: 0.9379 - loss: 0.2714

Epoch 15/20

3/3  0s 106ms/step - accuracy: 0.9262 - loss: 0.2541

Epoch 16/20

3/3  0s 114ms/step - accuracy: 0.9705 - loss: 0.2214

Epoch 17/20

3/3  0s 111ms/step - accuracy: 0.9596 - loss: 0.2150

Epoch 18/20

3/3  0s 111ms/step - accuracy: 0.9674 - loss: 0.1994

Epoch 19/20

3/3  0s 102ms/step - accuracy: 0.9627 - loss: 0.1935

Epoch 20/20

3/3 ————— 0s 98ms/step - accuracy: 1.0000 - loss: 0.1746

Evaluating DenseNet+LSTM model...

1/1 ————— 5s 5s/step - accuracy: 0.8000 - loss: 0.3752

DenseNet+LSTM Test Accuracy: 0.8000

```

from tensorflow import keras
from tensorflow.keras import layers

def build_vit_classifier(input_shape, num_classes, patch_size, num_patches, proj_dim, num_heads, dropout):
    inputs = layers.Input(shape=input_shape)

    patches = layers.Conv2D(filters=projection_dim, kernel_size=patch_size, strides=patch_size)(inputs)
    patches = layers.Reshape((num_patches, projection_dim))(patches)

    positions = tf.range(start=0, limit=num_patches, delta=1)
    position_embedding = layers.Embedding(input_dim=num_patches, output_dim=projection_dim)(positions)
    encoded_patches = patches + position_embedding

    for _ in range(transformer_layers):
        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
        attention_output = layers.MultiHeadAttention(num_heads=num_heads, key_dim=projection_dim)(x1, x1)
        x2 = layers.Add()([attention_output, encoded_patches])
        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = keras.Sequential([
            layers.Dense(mlp_hidden_units, activation="gelu"),
            layers.Dropout(dropout),
            layers.Dense(projection_dim),
            layers.Dropout(dropout),
        ])(x3)
        encoded_patches = layers.Add()([x3, x2])

    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)

    features = keras.Sequential([
        layers.Dense(mlp_hidden_units, activation="gelu"),
        layers.Dropout(dropout),
        layers.Dense(num_classes, activation="softmax")
    ])(representation)

    model = keras.Model(inputs=inputs, outputs=features)
    return model

```

```

input_shape = (224, 224, 3)
num_classes = 2
patch_size = 16
num_patches = (input_shape[0] // patch_size) * (input_shape[1] // patch_size)
projection_dim = 64
num_heads = 4
transformer_layers = 8
mlp_hidden_units = 128
dropout = 0.1

vit_model = build_vit_classifier(
    input_shape=input_shape,
    num_classes=num_classes,
    patch_size=patch_size,
    num_patches=num_patches,
    projection_dim=projection_dim,
    num_heads=num_heads,
    transformer_layers=transformer_layers,
    mlp_hidden_units=mlp_hidden_units,
    dropout=dropout
)

vit_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metr

print("Training Vision Transformer (ViT) model...")
vit_model.fit(train_ds, epochs=15)

print("Evaluating Vision Transformer (ViT) model...")
loss_vit, acc_vit = vit_model.evaluate(test_ds)
print(f'Vision Transformer (ViT) Test Accuracy: {acc_vit:.4f}')

```

```

Training Vision Transformer (ViT) model...
Epoch 1/15
3/3 ————— 79s 12s/step - accuracy: 0.5009 - loss: 3.8523
Epoch 2/15
3/3 ————— 0s 62ms/step - accuracy: 0.4588 - loss: 3.1540
Epoch 3/15
3/3 ————— 0s 69ms/step - accuracy: 0.4991 - loss: 1.1416
Epoch 4/15
3/3 ————— 0s 60ms/step - accuracy: 0.5421 - loss: 1.1382
Epoch 5/15
3/3 ————— 1s 48ms/step - accuracy: 0.5599 - loss: 0.9141
Epoch 6/15
3/3 ————— 0s 50ms/step - accuracy: 0.5195 - loss: 0.7843
Epoch 7/15
3/3 ————— 0s 51ms/step - accuracy: 0.5030 - loss: 0.7664
Epoch 8/15
3/3 ————— 0s 53ms/step - accuracy: 0.4900 - loss: 0.6918
Epoch 9/15
3/3 ————— 0s 46ms/step - accuracy: 0.4844 - loss: 0.6951
Epoch 10/15
3/3 ————— 0s 49ms/step - accuracy: 0.5000 - loss: 0.6949
Epoch 11/15
3/3 ————— 0s 44ms/step - accuracy: 0.5030 - loss: 0.6948
Epoch 12/15

```

```

3/3 ————— 0s 46ms/step - accuracy: 0.4223 - loss: 0.6900
Epoch 13/15
3/3 ————— 0s 49ms/step - accuracy: 0.4332 - loss: 0.7024
Epoch 14/15
3/3 ————— 0s 49ms/step - accuracy: 0.5451 - loss: 0.6967
Epoch 15/15
3/3 ————— 0s 48ms/step - accuracy: 0.5117 - loss: 0.6955
Evaluating Vision Transformer (ViT) model...
1/1 ————— 3s 3s/step - accuracy: 0.5000 - loss: 0.6966
Vision Transformer (ViT) Test Accuracy: 0.5000

```

```

from tensorflow.keras import layers, models

def build_deit_classifier(input_shape, num_classes, patch_size, num_patches, projection_dim):
    inputs = layers.Input(shape=input_shape)

    patches = layers.Conv2D(filters=projection_dim, kernel_size=patch_size, strides=patch_size)(inputs)
    patches = layers.Reshape((num_patches, projection_dim))(patches)

    positions = tf.range(start=0, limit=num_patches, delta=1)
    position_embedding = layers.Embedding(input_dim=num_patches, output_dim=projection_dim)(positions)
    encoded_patches = patches + position_embedding

    for _ in range(transformer_layers):

        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)

        attention_output = layers.MultiHeadAttention(num_heads=num_heads, key_dim=projection_dim)(x1, x1)
        x2 = layers.Add()([attention_output, encoded_patches])

        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)

        x3 = keras.Sequential([
            layers.Dense(mlp_hidden_units, activation="gelu"),
            layers.Dropout(dropout),
            layers.Dense(projection_dim),
            layers.Dropout(dropout),
        ])(x3)

        encoded_patches = layers.Add()([x3, x2])

    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)

    features = keras.Sequential([
        layers.Dense(mlp_hidden_units, activation="gelu"),
        layers.Dropout(dropout),
        layers.Dense(num_classes, activation="softmax")
    ])(representation)

    model = keras.Model(inputs=inputs, outputs=features)
    return model

```

```

input_shape = (224, 224, 3)
num_classes = 2
patch_size = 16
num_patches = (input_shape[0] // patch_size) * (input_shape[1] // patch_size)
projection_dim = 64
num_heads = 4
transformer_layers = 8
mlp_hidden_units = 128
dropout = 0.1

deit_model = build_deit_classifier(
    input_shape=input_shape,
    num_classes=num_classes,
    patch_size=patch_size,
    num_patches=num_patches,
    projection_dim=projection_dim,
    num_heads=num_heads,
    transformer_layers=transformer_layers,
    mlp_hidden_units=mlp_hidden_units,
    dropout=dropout
)

deit_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', met

print("Training Data-efficient Image Transformers (DeiT) model...")
deit_model.fit(train_ds, epochs=10) # Adjust epochs as needed

print("Evaluating Data-efficient Image Transformers (DeiT) model...")
loss_deit, acc_deit = deit_model.evaluate(test_ds)
print(f'Data-efficient Image Transformers (DeiT) Test Accuracy: {acc_deit:.4f}')

```

```

Training Data-efficient Image Transformers (DeiT) model...
Epoch 1/10
3/3 ————— 72s 12s/step - accuracy: 0.5078 - loss: 8.5581
Epoch 2/10
3/3 ————— 0s 56ms/step - accuracy: 0.5434 - loss: 2.7985
Epoch 3/10
3/3 ————— 0s 45ms/step - accuracy: 0.5846 - loss: 0.8923
Epoch 4/10
3/3 ————— 0s 45ms/step - accuracy: 0.5282 - loss: 0.7924
Epoch 5/10
3/3 ————— 0s 51ms/step - accuracy: 0.4939 - loss: 0.6938
Epoch 6/10
3/3 ————— 0s 46ms/step - accuracy: 0.3967 - loss: 0.6982
Epoch 7/10
3/3 ————— 0s 49ms/step - accuracy: 0.5000 - loss: 0.6939
Epoch 8/10
3/3 ————— 0s 47ms/step - accuracy: 0.4883 - loss: 0.6931
Epoch 9/10
3/3 ————— 0s 46ms/step - accuracy: 0.5030 - loss: 0.6932
Epoch 10/10
3/3 ————— 0s 47ms/step - accuracy: 0.5039 - loss: 0.6926
Evaluating Data-efficient Image Transformers (DeiT) model...
1/1 ————— 3s 3s/step - accuracy: 0.5000 - loss: 0.6931
Data-efficient Image Transformers (DeiT) Test Accuracy: 0.5000

```

```

def build_ast_image_classifier(input_shape, num_classes, patch_size, num_patches
    inputs = layers.Input(shape=input_shape)

    patches = layers.Conv2D(filters=projection_dim, kernel_size=patch_size, stri
    patches = layers.Reshape((num_patches, projection_dim))(patches)

    positions = tf.range(start=0, limit=num_patches, delta=1)
    position_embedding = layers.Embedding(input_dim=num_patches, output_dim=proj
    encoded_patches = patches + position_embedding

    for _ in range(transformer_layers):

        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)

        attention_output = layers.MultiHeadAttention(num_heads=num_heads, key_di

        x2 = layers.Add()([attention_output, encoded_patches])

        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)

        x3 = keras.Sequential([
            layers.Dense(mlp_hidden_units, activation="gelu"),
            layers.Dropout(dropout),
            layers.Dense(projection_dim),
            layers.Dropout(dropout),
        ])(x3)

        encoded_patches = layers.Add()([x3, x2])

    representation = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)

    features = keras.Sequential([
        layers.Dense(mlp_hidden_units, activation="gelu"),
        layers.Dropout(dropout),
        layers.Dense(num_classes, activation="softmax")
    ])(representation)

    model = keras.Model(inputs=inputs, outputs=features)
    return model

input_shape = (224, 224, 3)
num_classes = 2
patch_size = 16
num_patches = (input_shape[0] // patch_size) * (input_shape[1] // patch_size)
projection_dim = 64
num_heads = 4
transformer_layers = 8
mlp_hidden_units = 128
dropout = 0.1

ast_image_model = build_ast_image_classifier(
    input_shape=input_shape,

```

```

num_classes=num_classes,
patch_size=patch_size,
num_patches=num_patches,
projection_dim=projection_dim,
num_heads=num_heads,
transformer_layers=transformer_layers,
mlp_hidden_units=mlp_hidden_units,
dropout=dropout
)

ast_image_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

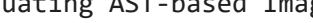
print("Training AST-based Image Classifier model...")
ast_image_model.fit(train_ds, epochs=10)

print("Evaluating AST-based Image Classifier model...")
loss_ast, acc_ast = ast_image_model.evaluate(test_ds)
print(f'AST-based Image Classifier Test Accuracy: {acc_ast:.4f}')

print("\n--- Model Accuracies ---")
print(f'Vision Transformer (ViT) Test Accuracy: {acc_vit:.4f}')
print(f'Data-efficient Image Transformers (DeiT) Test Accuracy: {acc_deit:.4f}')
print(f'AST-based Image Classifier Test Accuracy: {acc_ast:.4f}')

```

```

Training AST-based Image Classifier model...
Epoch 1/10
3/3  72s 13s/step - accuracy: 0.4232 - loss: 2.5908
Epoch 2/10
3/3  0s 67ms/step - accuracy: 0.4323 - loss: 1.5310
Epoch 3/10
3/3  0s 71ms/step - accuracy: 0.5234 - loss: 1.0352
Epoch 4/10
3/3  1s 76ms/step - accuracy: 0.4948 - loss: 0.8950
Epoch 5/10
3/3  0s 69ms/step - accuracy: 0.4831 - loss: 0.7234
Epoch 6/10
3/3  0s 64ms/step - accuracy: 0.4813 - loss: 0.7137
Epoch 7/10
3/3  0s 56ms/step - accuracy: 0.5312 - loss: 0.6895
Epoch 8/10
3/3  0s 54ms/step - accuracy: 0.4392 - loss: 0.7324
Epoch 9/10
3/3  0s 48ms/step - accuracy: 0.3750 - loss: 0.7351
Epoch 10/10
3/3  0s 52ms/step - accuracy: 0.4479 - loss: 0.7386
Evaluating AST-based Image Classifier model...
1/1  3s 3s/step - accuracy: 0.5000 - loss: 0.7019
AST-based Image Classifier Test Accuracy: 0.5000

--- Model Accuracies ---
Vision Transformer (ViT) Test Accuracy: 0.5000
Data-efficient Image Transformers (DeiT) Test Accuracy: 0.5000
AST-based Image Classifier Test Accuracy: 0.5000

```

```

def build_and_train_gwo_standalone_model(hyperparameters, train_ds, test_ds):
    """Builds, compiles, and trains a standalone GWO-optimized CNN model."""
    model = models.Sequential([

```



```

layers.Conv2D(hyperparameters['conv1_filters'], (3,3), activation='relu')
layers.MaxPooling2D((2,2)),
layers.Conv2D(hyperparameters['conv2_filters'], (3,3), activation='relu')
layers.MaxPooling2D((2,2)),
layers.Flatten(),
layers.Dense(hyperparameters['dense_units'], activation='relu'),
layers.Dropout(hyperparameters['dropout_rate']),
layers.Dense(2, activation='softmax')
])

optimizer = optimizers.Adam(learning_rate=hyperparameters['learning_rate'])

model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy', m

history = model.fit(train_ds, epochs=hyperparameters['epochs'], validation_d

loss, acc = model.evaluate(test_ds, verbose=0)
return acc, history.history['val_accuracy'][-1]

```

```

import tensorflow as tf
from tensorflow.keras.preprocessing import image_dataset_from_directory
import numpy as np
import os, cv2, zipfile

if os.path.exists("/content/spiral.zip"):
    with zipfile.ZipFile("/content/spiral.zip", 'r') as zip_ref:
        zip_ref.extractall("/content/spiral")
    print(" Extracted spiral.zip successfully!")

print("Root directory:", os.listdir('/content/spiral'))

if 'spiral' in os.listdir('/content/spiral'):
    dataset_path = '/content/spiral/spiral'
else:
    dataset_path = '/content/spiral'

print("Using dataset path:", dataset_path)
print("Subfolders:", os.listdir(dataset_path))

train_ds = image_dataset_from_directory(
    os.path.join(dataset_path, 'training'),
    image_size=(224, 224),
    batch_size=32
)
test_ds = image_dataset_from_directory(
    os.path.join(dataset_path, 'testing'),
    image_size=(224, 224),
    batch_size=32
)

print("Classes in DL loader:", train_ds.class_names)

train_ds = train_ds.map(lambda x, y: (x / 255.0, y))
test_ds = test_ds.map(lambda x, y: (x / 255.0, y))

```

```

def load_images(folder):
    images = []
    labels = []
    for label, subfolder in enumerate(['healthy', 'parkinson']):
        path = os.path.join(folder, subfolder)
        for file in os.listdir(path):
            img_path = os.path.join(path, file)
            if file.lower().endswith(('.png', '.jpg', '.jpeg')):
                img = cv2.imread(img_path)
                img = cv2.resize(img, (224, 224))
                img = img / 255.0 # Normalize
                images.append(img.flatten())
                labels.append(label)
    return np.array(images), np.array(labels)

X_train_ml, y_train_ml = load_images(os.path.join(dataset_path, 'training'))
X_test_ml, y_test_ml = load_images(os.path.join(dataset_path, 'testing'))

print(" Data Loaded Successfully")
print("Train shape:", X_train_ml.shape, "Labels:", y_train_ml.shape)
print("Test shape:", X_test_ml.shape, "Labels:", y_test_ml.shape)

from tensorflow.keras.applications import VGG16, ResNet50
from tensorflow.keras.layers import Concatenate, Input, Dense, GlobalAveragePool
from tensorflow.keras import Model

def build_dual_side_model(input_shape, num_classes):
    inputs = Input(shape=input_shape)

    base_model_vgg = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
    base_model_vgg.trainable = False
    x1 = base_model_vgg(inputs, training=False)
    x1 = GlobalAveragePooling2D()(x1)

    base_model_resnet = ResNet50(weights='imagenet', include_top=False, input_shape=input_shape)
    base_model_resnet.trainable = False
    x2 = base_model_resnet(inputs, training=False)
    x2 = GlobalAveragePooling2D()(x2)

    combined_features = Concatenate()([x1, x2])

    output = Dense(num_classes, activation='softmax')(combined_features)

    model = Model(inputs=inputs, outputs=output)
    return model

input_shape = (224, 224, 3)
num_classes = 2
dual_side_model = build_dual_side_model(input_shape, num_classes)

dual_side_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy')

```

```
print("\nTraining Deep dual-side learning ensemble model...")
dual_side_model.fit(train_ds, epochs=15)

print("Evaluating Deep dual-side learning ensemble model...")
loss_dual_side, acc_dual_side = dual_side_model.evaluate(test_ds)
print(f'Deep dual-side learning ensemble model Test Accuracy: {acc_dual_side:.4f}
```

```
    Extracted spiral.zip successfully!
    Root directory: ['spiral']
    Using dataset path: /content/spiral/spiral
    Subfolders: ['testing', 'training']
    Found 72 files belonging to 2 classes.
    Found 30 files belonging to 2 classes.
    Classes in DL loader: ['healthy', 'parkinson']
```

```
    Data Loaded Successfully
    Train shape: (72, 150528) Labels: (72,)
    Test shape: (30, 150528) Labels: (30,)
```

```
Training Deep dual-side learning ensemble model...
```

```
Epoch 1/15
```

```
3/3 ————— 17s 3s/step - accuracy: 0.5109 - loss: 0.7813
```

```
Epoch 2/15
```

```
3/3 ————— 1s 272ms/step - accuracy: 0.5443 - loss: 0.6973
```

```
Epoch 3/15
```

```
3/3 ————— 1s 169ms/step - accuracy: 0.5660 - loss: 0.6766
```

```
Epoch 4/15
```

```
3/3 ————— 1s 169ms/step - accuracy: 0.6406 - loss: 0.6506
```

```
Epoch 5/15
```

```
3/3 ————— 1s 170ms/step - accuracy: 0.5148 - loss: 0.6485
```

```
Epoch 6/15
```

```
3/3 ————— 1s 165ms/step - accuracy: 0.6133 - loss: 0.6252
```

```
Epoch 7/15
```

```
3/3 ————— 1s 166ms/step - accuracy: 0.7912 - loss: 0.6088
```

```
Epoch 8/15
```

```
3/3 ————— 1s 167ms/step - accuracy: 0.6879 - loss: 0.6054
```

```
Epoch 9/15
```

```
3/3 ————— 1s 169ms/step - accuracy: 0.7717 - loss: 0.5829
```

```
Epoch 10/15
```

```
3/3 ————— 1s 166ms/step - accuracy: 0.7943 - loss: 0.5662
```

```
Epoch 11/15
```

```
3/3 ————— 1s 170ms/step - accuracy: 0.7973 - loss: 0.5657
```

```
Epoch 12/15
```

```
3/3 ————— 1s 171ms/step - accuracy: 0.7587 - loss: 0.5578
```

```
Epoch 13/15
```

```
3/3 ————— 1s 171ms/step - accuracy: 0.8424 - loss: 0.5381
```

```
Epoch 14/15
```

```
3/3 ————— 1s 169ms/step - accuracy: 0.7843 - loss: 0.5247
```

```
Epoch 15/15
```

```
3/3 ————— 1s 169ms/step - accuracy: 0.7964 - loss: 0.5286
```

```
Evaluating Deep dual-side learning ensemble model...
```

```
1/1 ————— 5s 5s/step - accuracy: 0.8000 - loss: 0.5254
```

```
Deep dual-side learning ensemble model Test Accuracy: 0.8000
```

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
import time
```

```

def create_pretrained_model_tf(input_shape, num_classes):
    """Creates a model using a pre-trained MobileNetV2 model."""
    base_model = MobileNetV2(input_shape=input_shape,
                              include_top=False,
                              weights='imagenet')

    base_model.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(128, activation='relu')(x)
    predictions = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=predictions)
    return model

for images, labels in train_ds.take(1):
    input_shape = images.shape[1:]
    num_classes = len(tf.unique(labels)[0])

pretrained_tf_model = create_pretrained_model_tf(input_shape, num_classes)
pretrained_tf_model.compile(optimizer='adam',
                             loss='sparse_categorical_crossentropy',
                             metrics=['accuracy'])

print("\nTraining Pre-trained TensorFlow Model (MobileNetV2)...")
start_time_pretrained_tf = time.time()

history_pretrained_tf = pretrained_tf_model.fit(train_ds, epochs=20, validation_
end_time_pretrained_tf = time.time()
pretrained_tf_training_time = end_time_pretrained_tf - start_time_pretrained_tf
print(f"Pre-trained TensorFlow Model Training Time: {pretrained_tf_training_time

loss_pretrained_tf, acc_pretrained_tf = pretrained_tf_model.evaluate(test_ds, ve
print(f"Pre-trained TensorFlow Model (MobileNetV2) Test Accuracy: {acc_pretrained

```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/9406464/9406464> ————— 2s 0us/step

Training Pre-trained TensorFlow Model (MobileNetV2)...

Epoch 1/20

3/3 ————— 39s 11s/step - accuracy: 0.6506 - loss: 0.7431 - val_acc

Epoch 2/20

3/3 ————— 0s 94ms/step - accuracy: 0.6194 - loss: 0.6200 - val_acc

Epoch 3/20

3/3 ————— 0s 98ms/step - accuracy: 0.7452 - loss: 0.5064 - val_acc

Epoch 4/20

3/3 ————— 0s 92ms/step - accuracy: 0.8542 - loss: 0.3455 - val_acc

Epoch 5/20

3/3 ————— 0s 92ms/step - accuracy: 0.8563 - loss: 0.3475 - val_acc

Epoch 6/20

3/3 ————— 0s 118ms/step - accuracy: 0.9115 - loss: 0.2678 - val_acc

Epoch 7/20

3/3 ————— 0s 95ms/step - accuracy: 0.8424 - loss: 0.2843 - val_acc

Epoch 8/20

3/3 ————— 0s 91ms/step - accuracy: 0.9371 - loss: 0.2394 - val_acc

Epoch 9/20

3/3 ————— 0s 93ms/step - accuracy: 0.9488 - loss: 0.2309 - val_acc

```

Epoch 10/20
3/3 ————— 0s 101ms/step - accuracy: 0.9518 - loss: 0.1844 - val_acc
Epoch 11/20
3/3 ————— 0s 117ms/step - accuracy: 0.9440 - loss: 0.1744 - val_acc
Epoch 12/20
3/3 ————— 0s 139ms/step - accuracy: 0.9813 - loss: 0.1387 - val_acc
Epoch 13/20
3/3 ————— 1s 135ms/step - accuracy: 1.0000 - loss: 0.1256 - val_acc
Epoch 14/20
3/3 ————— 0s 155ms/step - accuracy: 0.9813 - loss: 0.1077 - val_acc
Epoch 15/20
3/3 ————— 0s 148ms/step - accuracy: 0.9813 - loss: 0.1070 - val_acc
Epoch 16/20
3/3 ————— 0s 94ms/step - accuracy: 0.9813 - loss: 0.0987 - val_acc
Epoch 17/20
3/3 ————— 0s 97ms/step - accuracy: 0.9891 - loss: 0.0839 - val_acc
Epoch 18/20
3/3 ————— 0s 117ms/step - accuracy: 0.9891 - loss: 0.0702 - val_acc
Epoch 19/20
3/3 ————— 0s 118ms/step - accuracy: 1.0000 - loss: 0.0672 - val_acc
Epoch 20/20
3/3 ————— 0s 98ms/step - accuracy: 1.0000 - loss: 0.0673 - val_acc
Pre-trained TensorFlow Model Training Time: 46.12 seconds
Pre-trained TensorFlow Model (MobileNetV2) Test Accuracy: 0.8667

```

```

import pandas as pd
model_accuracies = {}

if 'acc_inceptionv3' in globals():
    model_accuracies['InceptionV3'] = acc_inceptionv3
if 'acc_densenet121' in globals():
    model_accuracies['DenseNet121'] = acc_densenet121
if 'acc_resnet18' in globals():
    model_accuracies['ResNet18 (using ResNet50 base)'] = acc_resnet18
if 'acc_resnet101' in globals():
    model_accuracies['ResNet101 (using ResNet50 base)'] = acc_resnet101
if 'acc_vgg19' in globals():
    model_accuracies['VGG19'] = acc_vgg19
if 'acc_dual_side' in globals():
    model_accuracies['Deep Dual-Side Learning Ensemble Model'] = acc_dual_side
if 'acc_vit' in globals():
    model_accuracies['Vision Transformer (ViT)'] = acc_vit
if 'acc_deit' in globals():
    model_accuracies['Data-Efficient Image Transformer (DeiT)'] = acc_deit
if 'acc_ast' in globals():
    model_accuracies['AST-based Image Classifier'] = acc_ast
if 'acc_pretrained_tf' in globals():
    model_accuracies['Pre-trained TensorFlow Model (MobileNetV2)'] = acc_pretrained_tf

accuracy_df = pd.DataFrame.from_dict(model_accuracies, orient='index', columns=[
    'Test Accuracy'

accuracy_df = accuracy_df.sort_values(by='Test Accuracy', ascending=False)

print("Consolidated Model Accuracy Report:")

```

```
display(accuracy_df)

if not accuracy_df.empty:
    best_model = accuracy_df.iloc[0]
    print(f"\nBest Model: {best_model.name} – Accuracy: {best_model['Test Accuracy']}")
else:
    print("No model accuracies found. Please run model training first.")
```

Consolidated Model Accuracy Report:

	Test Accuracy
Pre-trained TensorFlow Model (MobileNetV2)	0.866667
DenseNet121	0.833333
Deep Dual-Side Learning Ensemble Model	0.800000
InceptionV3	0.766667
ResNet101 (using ResNet50 base)	0.600000
ResNet18 (using ResNet50 base)	0.533333
VGG19	0.533333
Vision Transformer (ViT)	0.500000
Data-Efficient Image Transformer (DeiT)	0.500000
AST-based Image Classifier	0.500000

Best Model: Pre-trained TensorFlow Model (MobileNetV2) – Accuracy: 0.8667

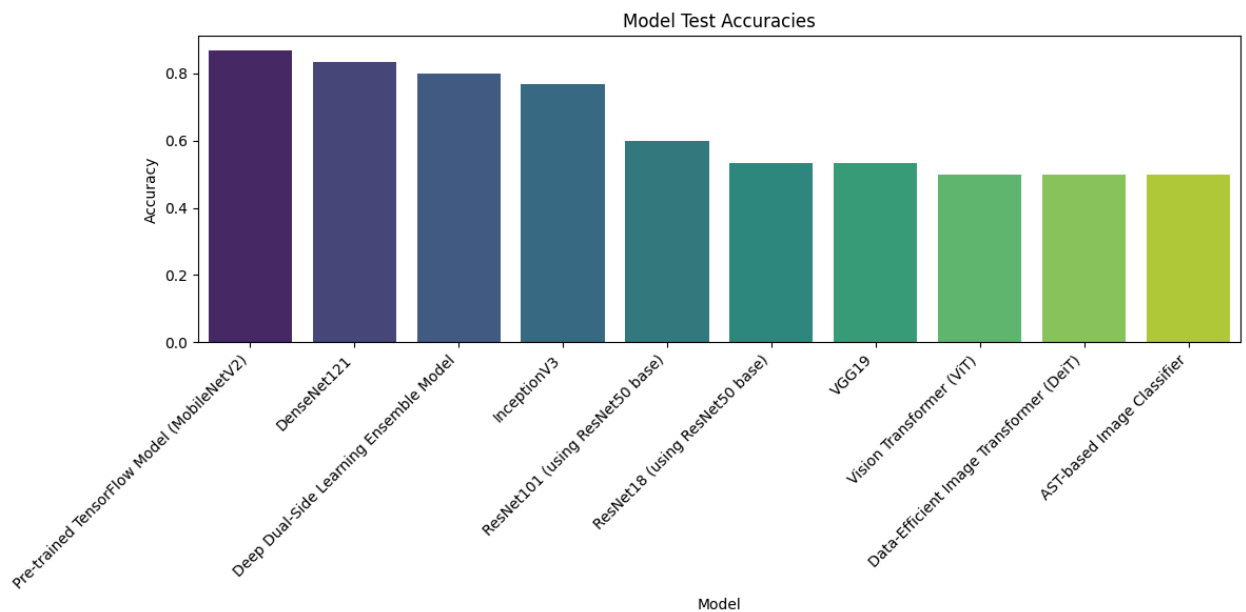
```
import matplotlib.pyplot as plt
import seaborn as sns

if not accuracy_df.empty:
    plt.figure(figsize=(12, 6))
    sns.barplot(x=accuracy_df.index, y='Test Accuracy', data=accuracy_df, palette='magma')
    plt.xticks(rotation=45, ha='right')
    plt.title('Model Test Accuracies')
    plt.ylabel('Accuracy')
    plt.xlabel('Model')
    plt.tight_layout()
    plt.show()
else:
    print("No model accuracies found to plot.")
```

/tmp/ipython-input-1849601250.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0

```
sns.barplot(x=accuracy_df.index, y='Test Accuracy', data=accuracy_df, palette='
```



```
best_model_name = accuracy_df.index[0]
print(f"The best performing model is: {best_model_name}")
```

The best performing model is: Pre-trained TensorFlow Model (MobileNetV2)

```
try:
    best_model_name = accuracy_df.index[0]
except NameError:
    print("Error: best_model_name not found. Please run the cell to identify th
except IndexError:
    print("Error: accuracy_df is empty. No model accuracies found.")

if 'best_model_name' in locals():
    if best_model_name == 'Pre-trained TensorFlow Model (MobileNetV2)':
        if 'pretrained_tf_model' in globals():
            pretrained_tf_model.save("MobileNet121.h5")
            print(f"Saved {best_model_name} to MobileNet121.h5")
        else:
            print(f"Error: Model object '{best_model_name}' not found.")
    elif best_model_name == 'DenseNet121':
        # Assuming acc_densenet121 was calculated from a model object named den
        # If the model object name is different, adjust accordingly
        if 'densenet121_model' in globals():
```

```

densenet121_model.save("MobileNet121.h5")
print(f"Saved {best_model_name} to MobileNet121.h5")
else:
    print(f"Error: Model object for '{best_model_name}' not found. Ple
elif best_model_name == 'InceptionV3':
    # Assuming acc_inceptionv3 was calculated from a model object named inc
    # If the model object name is different, adjust accordingly
    if 'inceptionv3_model' in globals():
        inceptionv3_model.save("MobileNet121.h5")
        print(f"Saved {best_model_name} to MobileNet121.h5")
    else:
        print(f"Error: Model object for '{best_model_name}' not found. Ple
elif best_model_name == 'Deep Dual-Side Learning Ensemble Model':
    if 'dual_side_model' in globals():
        dual_side_model.save("MobileNet121.h5")
        print(f"Saved {best_model_name} to MobileNet121.h5")
    else:
        print(f"Error: Model object '{best_model_name}' not found.")

# Add more elif conditions for other model names as needed

else:

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.save\_model()`. Saved Pre-trained TensorFlow Model (MobileNetV2) to MobileNet121.h5

```
from tensorflow.keras.models import load_model
```

```
loaded_model = load_model("MobileNet121.h5")
print("Model loaded successfully.")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. Model loaded successfully.

```
loss_loaded, acc_loaded = loaded_model.evaluate(test_ds, verbose=0)
print(f'Loaded Model Test Accuracy: {acc_loaded:.4f}')
```

Loaded Model Test Accuracy: 0.8667

```
!ls -l *.h5
```

```

-rw-r--r-- 1 root root 11543432 Oct 28 01:45 best_model.h5
-rw-r--r-- 1 root root 11543432 Oct 28 01:49 MobileNet121.h5

```